

Topic: Operations on Doubly Linked List

Q. Discuss the various operations (Traversal, Insertion, Deletion, Searching) on a Doubly Linked List. How does the presence of a 'previous' pointer change these operations compared to a Singly Linked List?

📌 Definition to Remember

A **Doubly Linked List (DLL)** is a linked data structure where each node contains three fields: data, a **next** pointer, and a **prev** (previous) pointer. The presence of the **prev** pointer allows for **bidirectional traversal** (forward and backward) and simplifies deletion operations.

Operations on Doubly Linked List

- **Forward Traversal:** Start at **head**, loop using **temp = temp->next**.
- **Backward Traversal:** Start at the last node (**tail**), loop using **temp = temp->prev**.
- **Searching:** Similar to a Singly Linked List, but if the location is known to be near the end, you can search backwards from the tail to save time.

2. Insertion Operations

In a DLL, both **next** and **prev** pointers must be carefully updated.

Position	Logic Steps
At Beginning	1. new_node->next = head; new_node->prev = NULL; 2. If list not empty: head->prev = new_node; 3. head = new_node;
At End	1. new_node->next = NULL; 2. Traverse to last node. 3. last->next = new_node; new_node->prev = last;
Middle (After Node)	1. new_node->next = temp->next; new_node->prev = temp; 2. temp->next->prev = new_node; 3. temp->next = new_node;

3. Deletion Operations

Deleting is highly efficient ($O(1)$) if a pointer to the target node (**del_node**) is already known, because we don't need to traverse from the head to find its preceding node.

Logic to delete **del_node :**

1. If deleting the head: **head = del_node->next;**
2. If it has a next node: **del_node->next->prev = del_node->prev;**
3. If it has a previous node: **del_node->prev->next = del_node->next;**
4. **free(del_node);**

4. Advantages vs Disadvantages

Detail	
Advantages	Bidirectional traversal; Deletion is $O(1)$ if node pointer is known; Easy to insert before a node.
Disadvantages	Extra memory required for <code>prev</code> pointer in every node; More pointer operations required for insert/delete.

★ Must-Write Points (for 10 marks)

1. Doubly Linked List has nodes with 3 fields: `prev` , `data` , and `next` .
2. Supports **bidirectional traversal** (forward using `next` , backward using `prev`).
3. Deletion of a known node is **$O(1)$** because the `prev` pointer gives instant access to the preceding node.
4. Insert at head: update `new_node->next` to head, and `head->prev` to new node.
5. Delete specific node: `del_node->next->prev = del_node->prev` and `del_node->prev->next = del_node->next` .
6. Requires more memory per node due to the extra `prev` pointer.
7. Used in applications requiring backward navigation (e.g., Browser history, undo operations).

⚡ Quick Recall

Doubly Linked List → `prev + data + next` → Bidirectional Traversal → Fast $O(1)$ Deletion (no need to find previous node) → Costs extra memory → Update both `prev` and `next` during insert/delete



Complete Executable C Code: Doubly Linked List Full Implementation

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *prev;
    struct Node *next;
};

// Insert at Head - O(1)
struct Node* insertHead(struct Node *head, int val) {
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = val;
    newNode->prev = NULL;
    newNode->next = head;

    if (head != NULL) head->prev = newNode;
    return newNode;
}

// Forward & Backward Traversal
void traverseForward(struct Node *head) {
    struct Node *temp = head;
    printf("Forward List: ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

void traverseBackward(struct Node *head) {
    if (head == NULL) return;
    struct Node *temp = head;
    while (temp->next != NULL) temp = temp->next;

    printf("Backward List: ");
    while (temp != NULL) {
        printf("%d <-> ", temp->data);
        temp = temp->prev;
    }
    printf("NULL\n");
}

int main() {
    struct Node *head = NULL;

    head = insertHead(head, 30);
    head = insertHead(head, 20);
    head = insertHead(head, 10);

    traverseForward(head);
    traverseBackward(head);
}
```

```
return 0;  
}
```